

vasic-digital

Vasic Digital

Held

AI-native Softwareentwicklung - gebaut, um Vertrauen zu verdienen.

Jeder kann eine App innerhalb eines Nachmittags mit einem LLM verknüpfen. Die eigentliche Herausforderung – der Teil, der entscheidet, ob ein AI-System nur eine Demo oder ein zuverlässiges Produkt ist – liegt in allem, was das Modell umgibt: die Provider-Abstraktion, die einen Ausfall übersteht, die Orchestrierung, die Agenten bei der Aufgabe hält, die Verifizierung, die ein Bluffen des Modells erkennt, und die Governance, die nachweist, dass das Ganze sich wie vorgesehen verhält. Genau das baut Vasic Digital. Wir entwerfen und liefern AI-Entwicklungssysteme – die Modelle, Agenten, Orchestrierung und Infrastruktur, die große Sprachmodelle in zuverlässige Software verwandeln – zusammen mit der Governance-Ebene, die sie ehrlich hält. Alles folgt einer unnachgiebigen Regel: Ein Feature ist nicht „fertig“, wenn die Tests bestanden sind; es ist fertig, wenn ein echter Nutzer es tatsächlich verwenden kann und es dokumentierte Beweise dafür gibt.

Über uns

Vasic Digital ist eine fokussierte Ingenieurspraxis, die eine vernetzte Familie von AI-Entwicklungsprodukten und wiederverwendbaren Modulen aufbaut. Statt eines Monolithen ist die Arbeit als Flotte organisiert: große Produktanwendungen, die auf Dutzenden kleinen, unabhängig getesteten und entkoppelten Modulen aufbauen – sodass bewährte Komponenten in jedem Produkt wiederverwendet werden, statt neu entwickelt zu werden. Die Hauptsprache ist **Go**, ergänzt durch **Kotlin / Kotlin Multiplatform, TypeScript/React, Python, Swift** und **Shell**, je nach Aufgabe ausgewählt: Go für Hochdurchsatz-Dienste und

Bibliotheken, Kotlin für Bereitstellungstools und plattformübergreifende Mobile-Entwicklung, TypeScript für typisierte Frontends, Python für AI/ML-Integration.

Was die Flotte zusammenhält, ist Disziplin, die nicht nur angestrebt, sondern mechanisch umgesetzt wird. Jedes Projekt erbt ein gemeinsames technisches **Constitution** als Git-Submodul – sodass eine einmal verschärfte Regel sich über eine Flotte von über 140 Repositories verbreitet – und jede Funktion, die ein Produkt verspricht, muss durch einen automatisierten, beweisgenerierenden Test abgesichert sein, bevor sie als ausgeliefert gilt. Das ist keine Marketingfloskel, die über die Arbeit gelegt wird; es ist das Betriebsmodell, nach dem die Arbeit funktioniert. Der kumulative Effekt ist der eigentliche Vorteil: Da generische Anforderungen in entkoppelten, unabhängig getesteten Modulen leben, landet eine Korrektur oder Verbesserung an einer Stelle und hebt sofort alle Produkte an – und jedes neue System wird aus Komponenten zusammengesetzt, die ihr Vertrauen bereits verdient haben.

Was wir tun

AI-basierte Entwicklung. Wir bauen das Fundament für AI-Systeme von Anfang bis Ende:

- **Multi-Provider-LLM-Zugriff** – eine First-Party-Abstraktion über 40+ Anbieter (Anthropic/Claude, OpenAI, DeepSeek, Gemini, Mistral, Cohere, Groq, xAI/Grok, Qwen, Perplexity, OpenRouter, Together AI, Replicate, Cerebras, Cloudflare Workers AI, SiliconFlow und lokale Ollama als Fallback) hinter einer einzigen Schnittstelle mit Wiederholungsversuchen, Stromkreisunterbrechern und Gesundheitsprüfungen.
- **Agenten-Orchestrierung** – Headless-CLI-Steuerungsebenen für Coding-Agenten, graphenbasierte agentische Workflows, mehrstufige „AI-Debatten“ für Konsensbildung und DAG/Pipeline-Laufzeitumgebungen.
- **LLM-Verifizierung** – eine Vertrauensebene, die Modelle mit einem obligatorischen Verständnis-Check („Siehst du meinen Code?“) sowie Latenz-, Streaming-, Funktionsaufruf-, Bild- und embeddings-Tests bewertet und eine verifizierte Konfiguration exportiert.
- **Abruf und Gedächtnis** – RAG, vector-Datenbanken, embeddings und fusionierte Agenten-Gedächtnis-Engines (Mem0 + Cognee + Letta) mit Komprimierung für unendlichen Kontext.
- **Defensive LLM** – Schutzmechanismen, PII-Erkennung, adversarische Red-Team-Tests und Eingabenormalisierung.

Die Helix-Produktfamilie. Unsere Flagship-Linie deckt den gesamten AI-Entwicklungszyklus ab:

- **HelixTrack** — eine frei zugängliche Alternative zu JIRA (dem Flagship der Helix-Track-Linie).
- **HelixAgent** — ein Ensemble-LLM-Dienst, der mehrere Modelle debattieren lässt und die Antwort ausliefert, auf die sie sich einigen.
- **HelixCode** — eine verteilte AI-Entwicklungsplattform, die Aufgaben auf SSH-gesteuerte Worker verteilt, mit Checkpoint/Rollback-Funktion.
- **HelixLLM** — ein einziges Binary, sechs Modi: OpenAI- und Anthropic-kompatible Inferenz von Laptop bis Cluster, über HTTP/3.
- **HelixCluster** — ein verteiltes Betriebssystem für AI-Computing, von Rechenzentrums-GPUs bis zu Edge-Handhelds.
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — die Provider-Abstraktion, die Agenten-Steuerungsebene und die verifizierte Wahrheitsquelle.
- **HelixMemory, HelixSkills, HelixSpecifier, HelixBuilder, HelixTranslate, HelixTerminator, HelixGitpx, HelixOTA, HelixPlay** — Speicherverwaltung, kontrollierte Fähigkeiten, spezifikationsgesteuerte Entwicklung, Anwendungsbau, verifizierte Übersetzung, Zero-Trust-Terminals, föderiertes Git, sichere OTA-Updates und selbstgehostetes Cloud-Gaming.

Tools und Utilities (vasic-digital utils). Eigenständige, produktionsreife Werkzeuge: **Catalogizer** (Multi-Protokoll, verschlüsseltes Medienverwaltungs-System), **Courses-Creator** (Markdown-zu-Video-AI-Kursproduktion), **VisionEngine** (Computer-Vision + LLM-basierte UI-Wahrnehmung), **DocProcessor** (Dokumentation-zu-Feature-Map für QS), **Docs Chain** (inhaltsbasierte bidirektionale Dokumenten-/Datenbank-Synchronisation), **Herald** (mehrkanalige Benachrichtigungen in natürlicher Sprache), **task_bridge** (bidirektionale Aufgaben-/Board-Synchronisation) und die **Vasic Digital Reusable Module Suite** — die `digital.vasic.*`-„Standardbibliothek“ für Infrastruktur, AI-Grundbausteine und Sicherheitsmodule.

Infrastrukturautomatisierung (Server Factory). Unser DevOps-Erbe: **Mail Server Factory** und das **Server Factory Core Framework**, die deklarative JSON in vollständig provisionierte, dockerisierte Server über verschiedene Verbindungstypen und Linux-Distributionen hinweg umwandeln, plus VM-Image-Tools (Qemu-Utills, Parallels-Utills) und unterstützende Service-Fabriken.

Technologien

Basierend auf unserem tatsächlichen Stack:

- **Sprachen:** Go (dominant), Kotlin & Kotlin Multiplatform, TypeScript, Python, Swift, Shell, mit PL/pgSQL und sogar TLA+-Formalismen in der Arbeit mit verteilten Systemen.
- **AI / LLM:** Multi-Provider-Zugriff (43+ Adapter), Model Context Protocol (MCP), RAG, vector-Datenbanken und embeddings, Planungsalgorithmen (HiPlan, MCTS, Tree of Thoughts), LLMops, Benchmarking (SWE-bench/HumanEval/MMLU) und TTS (Bark, SpeechT5).
- **Backend:** Gin (Go), gRPC + Protocol Buffers, HTTP/3 (QUIC), WebSockets, Angular- und React-Frontends, Kafka/RabbitMQ-Messaging.
- **Daten:** PostgreSQL, SQLite, SQLCipher (verschlüsselt im Ruhezustand), Redis, Neo4j, ClickHouse sowie Objektspeicher (MinIO/S3/GCS/Azure).
- **Infrastruktur / DevOps:** Docker & Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels sowie CI/CD über GitHub Actions, Gradle und Make.
- **Testing / QS:** das Anti-Bluff-HelixQA-Framework, modulare Challenge-Harnesses mit Mutations-Gates, `go test -race`, visuelle Regressions-Tools, ADB-Gerätetests, SonarQube-Gates sowie Sicherheits-Scans (semgrep, gosec, trivy, snyk, gitleaks, nancy).

Qualität & Steuerung — unser Alleinstellungsmerkmal

Zwei Säulen sorgen für Kohärenz und Vertrauenswürdigkeit der gesamten Flotte:

- **HelixConstitution** — ein universelles, projektunabhängiges Ingenieursregelwerk, das als Git-Submodul ausgeliefert und von jedem Projekt innerhalb einer Flotte von über 140 Repositories übernommen wird. Es kodifiziert unabdingbare Disziplin — Anti-Bluff-Evidenzprüfungen, Immunität gegen Fehllarme, Datensicherheit und Host-Schutz, Dokumentations- und Testabdeckungsvorgaben —, die ein Projekt erweitern, aber niemals abschwächen darf. Ein einziger Submodul-Update aktualisiert die Regeln überall; Propagationsprüfungen durchsuchen die gesamte Flotte per *grep* nach den erforderlichen Klauseln, und jede Prüfung ist mit einem Mutationstest gekoppelt, der beweist, dass die Prüfung selbst kein Placebo ist. Steuerung wird zur auditierbaren Tatsache, nicht zu einer bloßen Absichtserklärung.

- **HelixQA** — Anti-Bluff-Qualitätssicherungs-Orchestrierung. Sie führt schriftliche **YAML**-Testbanken sowie vollautonome **LLM**-plus-Computer-Vision-QS-Sitzungen auf Android, Android TV, Web und Desktop durch und verweigert die Bewertung „BESTANDEN“, solange keine Laufzeitbeweise (Screenshots, Logcat, Video, Stack Traces) vorliegen. „Wir haben es getestet“ wird zu „Hier sind das Video, das Logcat und das Ticket.“

Positionierung

Jeder kann eine App mit **LLM** verbinden. **Vasic Digital** baut den schwierigen Teil: **AI**-Systeme, die überprüfbar, wiederverwendbar und ehrlich sind — ein anbieterunabhängiges **AI**-Fundament, ein Lebenszyklus von **Helix**-Produkten darauf und eine Verfassung-plus-Evidenz-Disziplin, die garantiert, dass das, was ausgeliefert wird, auch wirklich funktioniert. Wir bitten Sie nicht, dem grünen Häkchen zu vertrauen. Wir zeigen Ihnen die Beweise dahinter.

Kontakt

Lassen Sie uns etwas Überprüfbares schaffen.

- **E-Mail:** i@mvasic.ru
- **GitHub:** github.com/vasic-digital